Lekcja 02: Środowisko pracy dla Data Science

`#lekcja` `#datascience` `#środowisko-pracy` `#jupyter` `#google-colab` `#python-setup` `#dev-environment`

W tej lekcji poznasz narzędzia i środowiska, które są fundamentem pracy każdego Data Scientist. Nauczysz się korzystać z Google Colab i Jupyter Notebook, zainstalować Python lokalnie oraz organizować projekty zgodnie z najlepszymi praktykami branżowymi. Szczególny nacisk położymy na efektywne wykorzystanie zasobów i aspekty środowiskowe.

---

1. Google Colab – chmurowe środowisko dla Data Science

Definition

Google Colab (Colaboratory) to darmowe środowisko notebookowe działające w chmurze Google, które pozwala pisać i wykonywać kod Python bezpośrednio w przeglądarce, bez konieczności instalacji lokalnej. Oferuje dostęp do darmowych zasobów obliczeniowych (CPU, GPU, TPU).

Wprowadzenie

Google Colab to jedno z najpopularniejszych narzędzi dla początkujących Data Scientistów. Główne zalety:

- **Brak konieczności instalacji** – wszystko działa w przeglądarce
- **Darmowy dostęp do GPU/TPU** – idealne do uczenia modeli Deep Learning
- **Integracja z Google Drive** – łatwe zapisywanie i udostępnianie notatników
- **Pre-instalowane biblioteki** – NumPy, Pandas, Matplotlib, TensorFlow już zainstalowane
- **Współpraca w czasie rzeczywistym** – możliwość pracy zespołowej

Przykład 1: Pierwsze kroki w Google Colab

```
# Sprawdźmy, jakie zasoby są dostępne w Google Colab
import sys
import psutil
import platform

# Informacje o systemie
```

```
print("=== INFORMACJE O SYSTEMIE ===")
print(f"System operacyjny: {platform.system()} {platform.release()}")
print(f"Python version: {sys.version}")
print()

# Informacje o pamięci RAM
print("=== PAMIĘĆ RAM ===")
ram_gb = psutil.virtual_memory().total / (1024**3)
ram_available_gb = psutil.virtual_memory().available / (1024**3)
print(f"Całkowita RAM: {ram_gb:.2f} GB")
print(f"Dostępna RAM: {ram_available_gb:.2f} GB")
print()

# Informacje o CPU
print("=== PROCESOR ===")
print(f"Liczba rdzeni CPU: {psutil.cpu_count()}")
print(f"Obciążenie CPU: {psutil.cpu_percent(interval=1)}%")
print()

# Sprawdzenie dostępności GPU (w Colab)
try:
    import torch
    if torch.cuda.is_available():
        print("=== GPU ===")
        print(f"GPU dostępne: Tak")
        print(f"Nazwa GPU: {torch.cuda.get_device_name(0)}")
        print(f"Pamięć GPU:
{torch.cuda.get_device_properties(0).total_memory / (1024**3):.2f} GB")
    else:
        print("GPU: Niedostępne (włącz GPU w Runtime -> Change runtime
type)")
except ImportError:
    print("PyTorch nie zainstalowany - nie można sprawdzić GPU")
```

Tip

W Google Colab zawsze sprawdzaj dostępne zasoby przed rozpoczęciem trenowania modeli. Limity darmowego planu mogą się różnić w zależności od obciążenia serwerów Google.

Przykład 2: Montowanie Google Drive i praca z plikami

```
# Montowanie Google Drive do Colab
from google.colab import drive
```

```python
import os

# Montujemy dysk (pojawi się prośba o autoryzację)
drive.mount('/content/drive')

# Sprawdzamy, czy dysk został zamontowany
if os.path.exists('/content/drive/MyDrive'):
    print("✓ Google Drive pomyślnie zamontowany!")

    # Tworzymy folder dla naszego projektu Data Science
    project_folder = '/content/drive/MyDrive/DataScience_Projects'
    os.makedirs(project_folder, exist_ok=True)
    print(f"✓ Folder projektu: {project_folder}")

    # Wyświetlamy zawartość folderu
    print("\nZawartość folderu MyDrive:")
    for item in os.listdir('/content/drive/MyDrive')[:10]:  # Pierwsze 10
elementów
        print(f"  - {item}")
else:
    print("✗ Problem z montowaniem Google Drive")

# Przykład zapisu danych do Google Drive
import pandas as pd
import numpy as np

# Tworzymy przykładowy dataset
data = {
    'session_id': range(1, 101),
    'cpu_usage': np.random.uniform(10, 90, 100),
    'ram_usage': np.random.uniform(20, 80, 100),
    'gpu_usage': np.random.uniform(0, 100, 100)
}
df = pd.DataFrame(data)

# Zapisujemy do Google Drive
save_path = f"{project_folder}/resource_usage.csv"
df.to_csv(save_path, index=False)
print(f"\n✓ Dataset zapisany: {save_path}")
print(f"Rozmiar pliku: {os.path.getsize(save_path)} bajtów")
```

> ⚠ **Warning**
>
> **Pamiętaj!** Sesja w Google Colab ma ograniczony czas działania (około 12 godzin dla darmowego planu). Po rozłączeniu wszystkie pliki NIE zapisane w Google Drive zostaną

utracone!

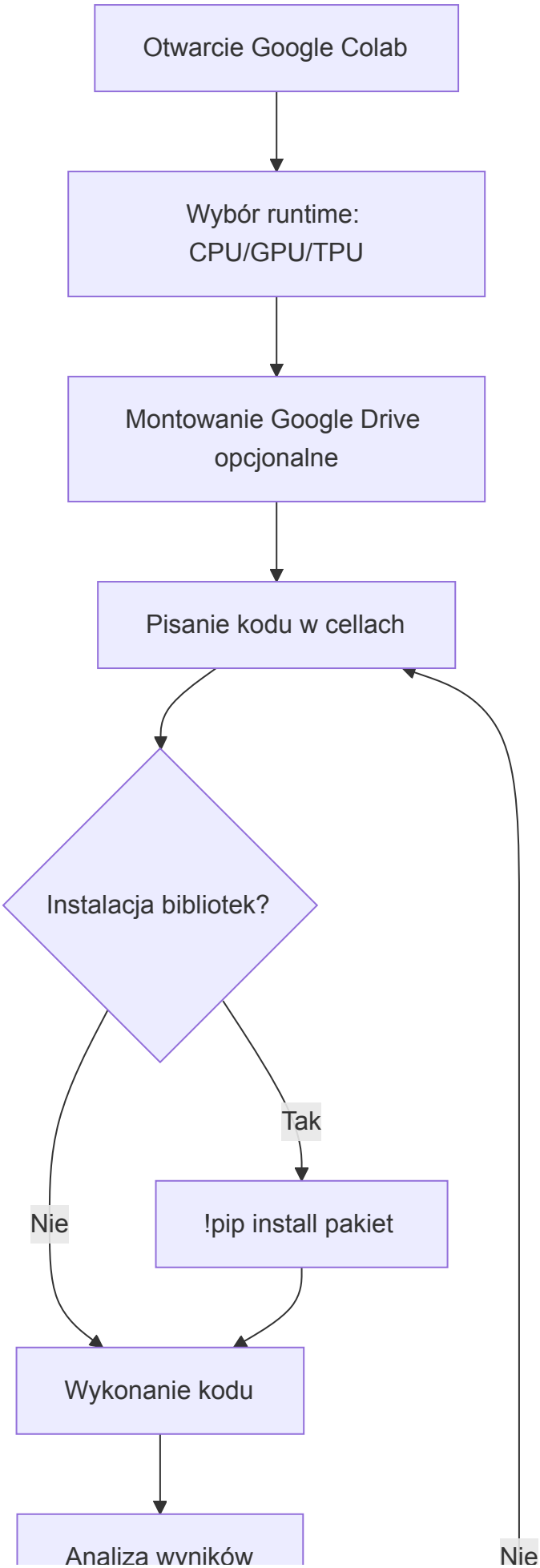
Przykład 3: Instalacja dodatkowych bibliotek w Colab

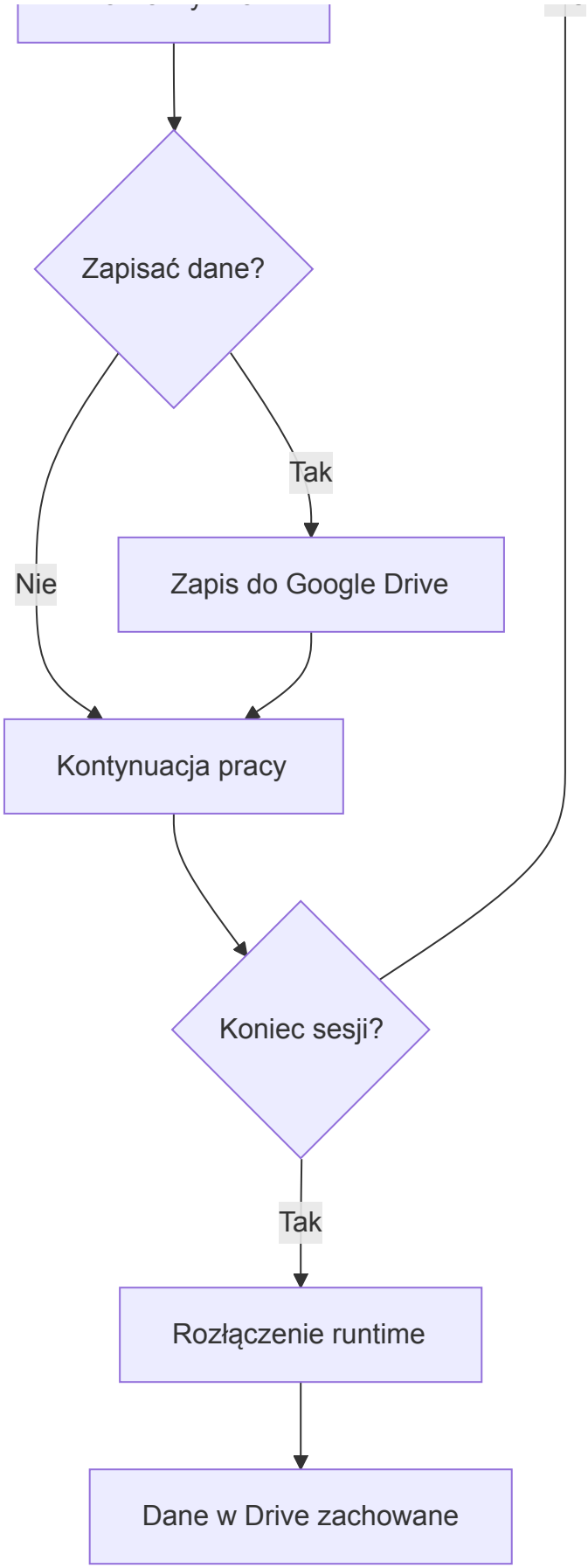
```
# Google Colab ma wiele pre-instalowanych bibliotek,
# ale czasem potrzebujemy czegoś specjalnego

# Sprawdzamy wersje już zainstalowanych bibliotek
import pandas as pd
import numpy as np
import matplotlib
import sklearn

print("=== PRE-INSTALOWANE BIBLIOTEKI ===")
print(f"Pandas: {pd.__version__}")
print(f"NumPy: {np.__version__}")
print(f"Matplotlib: {matplotlib.__version__}")
print(f"Scikit-learn: {sklearn.__version__}")

# Instalacja nowej biblioteki (przykład: seaborn dla zaawansowanych
wizualizacji)
!pip install -q seaborn  # -q oznacza "quiet mode" - mniej logów

import seaborn as sns
print(f"\n✓ Seaborn zainstalowany: {sns.__version__}")

# Instalacja biblioteki do monitorowania zużycia zasobów
!pip install -q memory_profiler

print("\n✓ Wszystkie wymagane biblioteki zainstalowane!")

# UWAGA: Instalacja w Colab działa tylko dla bieżącej sesji
# Po restarcie runtime trzeba ponownie zainstalować biblioteki
```

Note

W Google Colab używamy `!pip install` (z wykrzyknikiem) zamiast zwykłego `pip install` w terminalu. Wykrzyknik oznacza wykonanie komendy shell.

Wizualizacja workflow w Google Colab


Otwarcie Google Colab
Wybór runtime: CPU/GPU/TPU
Montowanie Google Drive opcjonalne
Pisanie kodu w cellach
Instalacja bibliotek?
Tak
Nie
!pip install pakiet
Wykonanie kodu
Analiza wyników
Nie



Best practices dla Google Colab

```python
# 1. ZAWSZE zapisuj ważne wyniki do Google Drive
# 2. Regularnie zapisuj notatnik (Ctrl+S)
# 3. Monitoruj zużycie zasobów
# 4. Zamykaj niepotrzebne sesje

# Przykład: funkcja do automatycznego zapisu wyników
import json
from datetime import datetime

def save_results_to_drive(results_dict, experiment_name):
    """
    Zapisuje wyniki eksperymentu do Google Drive z timestampem

    Args:
        results_dict: Słownik z wynikami eksperymentu
        experiment_name: Nazwa eksperymentu
    """
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename =
f"/content/drive/MyDrive/DataScience_Projects/results_{experiment_name}_{tim
estamp}.json"

    with open(filename, 'w') as f:
        json.dump(results_dict, f, indent=2)

    print(f"✓ Wyniki zapisane: {filename}")
    return filename

# Przykład użycia
experiment_results = {
    'model': 'LinearRegression',
    'accuracy': 0.87,
    'mse': 0.13,
    'training_time': '5.2s'
}

# save_results_to_drive(experiment_results, 'first_model')
# (odkomentuj gdy Drive jest zamontowany)
```

✓ **Success**

Green IT w Colab: Zawsze zamykaj sesję po zakończeniu pracy! Niepotrzebnie działająca sesja zużywa energię serwerów Google. Kliknij: Runtime → Disconnect and delete runtime.

---

2. Jupyter Notebook – lokalne środowisko programistyczne

Definition

Jupyter Notebook to open-source'owa aplikacja webowa pozwalająca tworzyć i udostępniać dokumenty zawierające kod na żywo, równania, wizualizacje i tekst narracyjny. Jest to standard w świecie Data Science do eksploracyjnej analizy danych i prototypowania.

Wprowadzenie

Jupyter Notebook różni się od Google Colab tym, że działa lokalnie na Twoim komputerze. Zalety:

- **Pełna kontrola nad środowiskiem** – instalujesz dokładnie to, czego potrzebujesz
- **Praca offline** – nie wymaga połączenia internetowego
- **Integracja z lokalnymi narzędziami** – łatwy dostęp do plików, baz danych, etc.
- **Brak limitów czasowych** – sesja działa tak długo, jak chcesz
- **Prywatność** – dane nie opuszczają Twojego komputera

Przykład 1: Instalacja Pythona i Jupyter Notebook

```
# === INSTALACJA PYTHON I JUPYTER (TERMINAL/CMD) ===

# KROK 1: Sprawdź, czy Python jest już zainstalowany
python --version
# lub
python3 --version

# Jeśli Python nie jest zainstalowany:
# - Windows: Pobierz z https://www.python.org/downloads/
# - macOS: brew install python3
# - Linux: sudo apt-get install python3
```

```
# KROK 2: Zaktualizuj pip (menedżer pakietów Pythona)
python -m pip install --upgrade pip

# KROK 3: Zainstaluj Jupyter Notebook
pip install jupyter

# KROK 4: Uruchom Jupyter Notebook
jupyter notebook
# Otworzy się przeglądarka z interfejsem Jupyter

# KROK 5: Sprawdź wersję
jupyter --version
```

```
# === W JUPYTER NOTEBOOK ===
# Po uruchomieniu jupyter notebook, utwórz nowy notatnik (New -> Python 3)

# Sprawdźmy, że wszystko działa
import sys
import jupyter

print("✓ Jupyter Notebook działa!")
print(f"Python: {sys.version}")
print(f"Jupyter core: {jupyter.__version__}")

# Sprawdzenie środowiska
import os
print(f"\nBieżący katalog roboczy: {os.getcwd()}")
print(f"Użytkownik: {os.getlogin()}")
```

Przykład 2: Praca z wirtualnymi środowiskami (venv)

```
# === TWORZENIE WIRTUALNEGO ŚRODOWISKA ===

# Wirtualne środowisko izoluje pakiety projektu od systemu
# To kluczowa praktyka w Data Science!

# KROK 1: Stwórz folder dla projektu
mkdir my_datascience_project
cd my_datascience_project

# KROK 2: Utwórz wirtualne środowisko
python -m venv ds_env
# lub
python3 -m venv ds_env
```

```
# KROK 3: Aktywuj środowisko

# Windows (CMD):
ds_env\Scripts\activate.bat

# Windows (PowerShell):
ds_env\Scripts\Activate.ps1

# macOS/Linux:
source ds_env/bin/activate

# Po aktywacji zobaczysz (ds_env) przed nazwą ścieżki

# KROK 4: Zainstaluj pakiety w środowisku
pip install jupyter pandas numpy matplotlib scikit-learn

# KROK 5: Utwórz plik requirements.txt
pip freeze > requirements.txt
# Ten plik zawiera listę wszystkich zainstalowanych pakietów

# KROK 6: Uruchom Jupyter z tego środowiska
jupyter notebook

# KROK 7: Dezaktywacja środowiska (po zakończeniu pracy)
deactivate
```

```
# === WERYFIKACJA ŚRODOWISKA W JUPYTER ===

import sys
import os

# Sprawdzamy, czy pracujemy w wirtualnym środowisku
print("=== ŚRODOWISKO PYTHON ===")
print(f"Python executable: {sys.executable}")
print(f"Python version: {sys.version}")

# Jeśli ścieżka zawiera 'ds_env' lub inną nazwę venv - jesteś w środowisku!
if 'ds_env' in sys.executable or 'venv' in sys.executable:
    print("\n✓ Pracujesz w wirtualnym środowisku!")
else:
    print("\n⚠ UWAGA: Prawdopodobnie używasz systemowego Pythona")

# Lista zainstalowanych pakietów
import pkg_resources
```

```
installed_packages = [d.project_name for d in pkg_resources.working_set]
print(f"\nZainstalowane pakiety: {len(installed_packages)}")
print("Przykładowe pakiety:")
for pkg in sorted(installed_packages)[:10]:
    print(f"  - {pkg}")
```

Tip

Dlaczego wirtualne środowisko?

- Różne projekty mogą wymagać różnych wersji tych samych bibliotek
- Unikasz konfliktów między pakietami
- Łatwo replikujesz środowisko na innym komputerze (przez requirements.txt)
- Nie zaśmiecasz systemowego Pythona

Przykład 3: Struktura projektu Data Science

```
# === TWORZENIE STRUKTURY PROJEKTU ===

import os

# Standardowa struktura projektu Data Science
project_structure = {
    'data': ['raw', 'processed', 'external'],
    'notebooks': ['exploratory', 'reports'],
    'src': ['data_processing', 'models', 'visualization'],
    'models': ['trained', 'experiments'],
    'reports': ['figures', 'tables'],
    'tests': [],
    'docs': []
}

def create_project_structure(project_name, structure):
    """
    Tworzy standardową strukturę folderów dla projektu DS

    Args:
        project_name: Nazwa projektu (nazwa głównego folderu)
        structure: Słownik ze strukturą folderów
    """
    # Tworzenie głównego folderu
    os.makedirs(project_name, exist_ok=True)
    print(f"✓ Utworzono projekt: {project_name}/")
```

```python
    # Tworzenie podfolderów
    for folder, subfolders in structure.items():
        folder_path = os.path.join(project_name, folder)
        os.makedirs(folder_path, exist_ok=True)
        print(f"  ├── {folder}/")

        for subfolder in subfolders:
            subfolder_path = os.path.join(folder_path, subfolder)
            os.makedirs(subfolder_path, exist_ok=True)
            print(f"  │   ├── {subfolder}/")

    # Tworzenie plików konfiguracyjnych
    files_to_create = {
        'README.md': f'# {project_name}\n\nOpis projektu Data Science.',
        '.gitignore':
'data/raw/\nmodels/trained/\n*.pyc\n__pycache__/\n.ipynb_checkpoints/',
        'requirements.txt':
'pandas>=1.5.0\nnumpy>=1.23.0\nmatplotlib>=3.6.0\nscikit-learn>=1.2.0',
        'config.py': '# Konfiguracja projektu\nDATA_PATH =
"data/"\nMODEL_PATH = "models/"'
    }

    for filename, content in files_to_create.items():
        filepath = os.path.join(project_name, filename)
        with open(filepath, 'w') as f:
            f.write(content)
        print(f"  ├── {filename}")

    print(f"\n✓ Struktura projektu '{project_name}' utworzona!")

# Użycie
# create_project_structure('energy_consumption_analysis', project_structure)
```

Przykładowa struktura projektu:

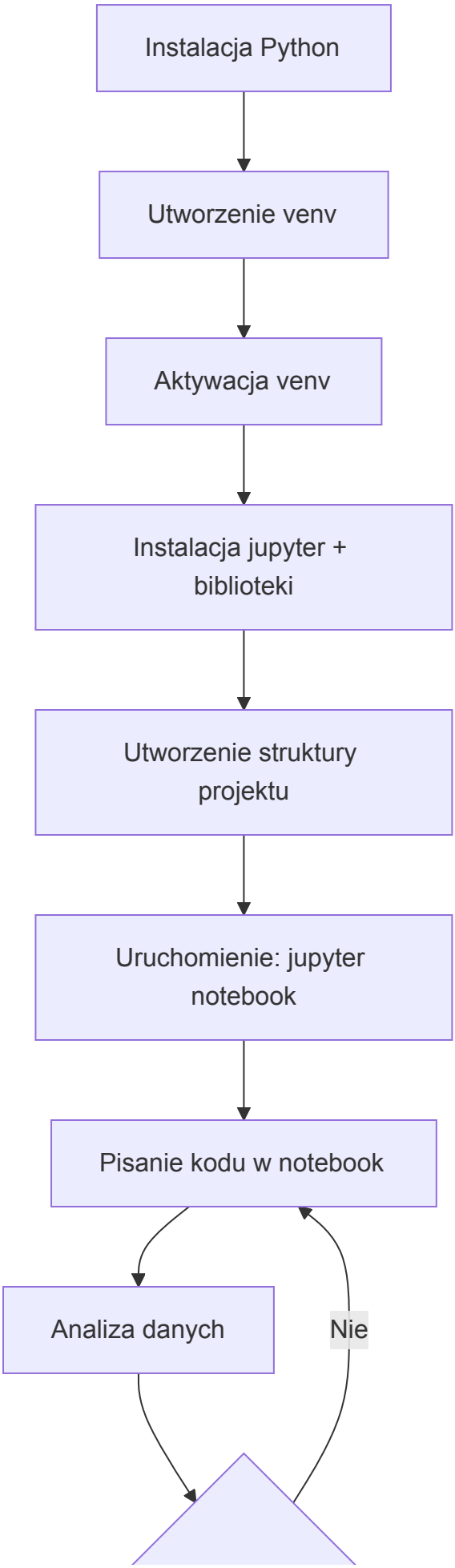
```
energy_consumption_analysis/
├── data/                    # Wszystkie dane
│   ├── raw/                # Surowe dane (NIGDY nie modyfikuj!)
│   ├── processed/          # Przetworzone, czyste dane
│   └── external/           # Dane z zewnętrznych źródeł
├── notebooks/              # Jupyter notebooks
│   ├── exploratory/        # Eksploracja danych (EDA)
│   └── reports/            # Finalne raporty
├── src/                    # Kod źródłowy
│   ├── data_processing/    # Skrypty do przetwarzania danych
```

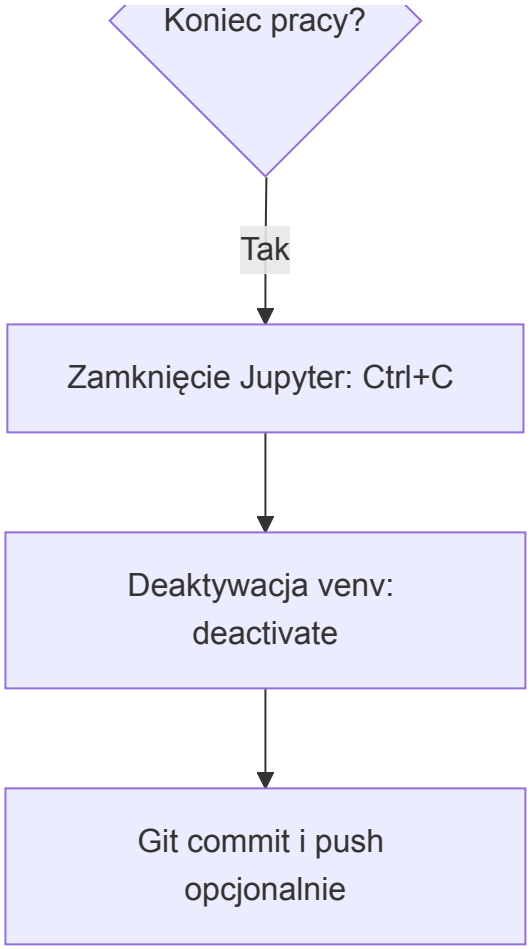
```
│   ├── models/             # Kod modeli ML
│   └── visualization/      # Funkcje do wizualizacji
├── models/                 # Wytrenowane modele
│   ├── trained/            # Gotowe modele
│   └── experiments/        # Eksperymentalne modele
├── reports/                # Raporty i wyniki
│   ├── figures/            # Wykresy, obrazy
│   └── tables/             # Tabele wyników
├── tests/                  # Testy jednostkowe
├── docs/                   # Dokumentacja
├── README.md               # Opis projektu
├── requirements.txt        # Lista zależności
├── .gitignore             # Pliki ignorowane przez git
└── config.py              # Konfiguracja
```

✓ **Success**

Ta struktura jest standardem w branży Data Science i pomaga utrzymać porządek w projektach!

Vizualizacja workflow Jupyter Notebook lokalnie


Instalacja Python
Utworzenie venv
Aktywacja venv
Instalacja jupyter + biblioteki
Utworzenie struktury projektu
Uruchomienie: jupyter notebook
Pisanie kodu w notebook
Analiza danych
Nie



---

3. Dobre praktyki pracy z notebookami

Definition

Dobre praktyki w Data Science to zbiór sprawdzonych zasad i konwencji, które pomagają tworzyć czytelny, reprodukowalny i łatwy w utrzymaniu kod analityczny.

Wprowadzenie

Jupyter Notebooks są świetne do eksploracji, ale łatwo mogą zamienić się w chaos. Oto zasady profesjonalnej pracy:

Przykład 1: Organizacja komórek w notebooku

```
# === SZABLON STRUKTURY JUPYTER NOTEBOOK ===

"""
TYTUŁ NOTATNIKA: Analiza zużycia energii przez modele ML
AUTOR: [Twoje imię]
DATA: 2025-11-21
```

```python
OPIS: Badanie wpływu wielkości modelu na zużycie energii podczas trenowania
"""

# ==========================================
# 1. IMPORTY
# ==========================================

# Standardowe biblioteki
import os
import sys
from datetime import datetime

# Analiza danych
import pandas as pd
import numpy as np

# Wizualizacja
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Konfiguracja wyświetlania
pd.set_option('display.max_columns', 50)
pd.set_option('display.max_rows', 100)
plt.style.use('seaborn-v0_8-darkgrid')

print("✓ Wszystkie biblioteki zaimportowane")
print(f"Data uruchomienia: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

# ==========================================
# 2. KONFIGURACJA
# ==========================================

# Ścieżki do danych
DATA_PATH = 'data/raw/'
PROCESSED_DATA_PATH = 'data/processed/'
MODELS_PATH = 'models/'
FIGURES_PATH = 'reports/figures/'

# Parametry analizy
RANDOM_STATE = 42
TEST_SIZE = 0.2
```

```python
# Ustawienia wizualizacji
FIGURE_SIZE = (12, 6)
DPI = 100

print("✓ Konfiguracja załadowana")

# ============================================
# 3. FUNKCJE POMOCNICZE
# ============================================

def load_and_validate_data(filepath):
    """
    Wczytuje dane i wykonuje podstawową walidację

    Args:
        filepath: Ścieżka do pliku CSV

    Returns:
        pd.DataFrame: Wczytane dane
    """
    df = pd.read_csv(filepath)

    print(f"✓ Dane wczytane: {df.shape[0]} wierszy, {df.shape[1]} kolumn")
    print(f"Brakujące wartości: {df.isnull().sum().sum()}")

    return df

def save_figure(fig, filename):
    """Zapisuje figurę do folderu reports/figures/"""
    os.makedirs(FIGURES_PATH, exist_ok=True)
    filepath = os.path.join(FIGURES_PATH, filename)
    fig.savefig(filepath, dpi=DPI, bbox_inches='tight')
    print(f"✓ Wykres zapisany: {filepath}")

print("✓ Funkcje pomocnicze zdefiniowane")

# ============================================
# 4. WCZYTANIE DANYCH
# ============================================

# Tu wczytasz dane - przykład poniżej
# df = load_and_validate_data(DATA_PATH + 'energy_data.csv')

# ============================================
# 5. EKSPLORACYJNA ANALIZA DANYCH (EDA)
# ============================================
```

```
# Tu przeprowadzasz analizę eksploracyjną

# ============================================
# 6. PRZETWARZANIE DANYCH
# ============================================

# Tu czyścisz i przygotowujesz dane

# ============================================
# 7. MODELOWANIE
# ============================================

# Tu tworzysz i trenujesz modele

# ============================================
# 8. EWALUACJA
# ============================================

# Tu oceniasz wyniki modeli

# ============================================
# 9. WNIOSKI
# ============================================

# Tu podsumowujesz wyniki
```

Note

Ta struktura sprawia, że każdy (w tym Ty za miesiąc) może łatwo zrozumieć, co dzieje się w notebooku!

Przykład 2: Czytelny kod w analizie danych

```
# === PRZYKŁAD: CZYTELNA ANALIZA EKSPLORACYJNA ===

# Tworzymy przykładowy dataset o zużyciu zasobów
np.random.seed(42)

data = {
    'model_size_mb': np.random.uniform(1, 500, 100),
    'training_time_min': np.random.uniform(5, 180, 100),
    'cpu_usage_percent': np.random.uniform(30, 100, 100),
    'ram_usage_gb': np.random.uniform(2, 32, 100),
```

```
    'accuracy': np.random.uniform(0.7, 0.99, 100)
}

df = pd.DataFrame(data)

# Dodajemy zużycie energii (symulacja: większy model = więcej energii)
df['energy_kwh'] = (
    df['training_time_min'] / 60 *  # godziny
    (df['cpu_usage_percent'] / 100 * 0.3 +  # zużycie CPU
     df['ram_usage_gb'] / 32 * 0.1)  # zużycie RAM
)

print("=== PODSTAWOWE STATYSTYKI ===")
print(df.describe())

# ❌ ZŁY PRZYKŁAD: Nieczytelny kod
# plt.figure(figsize=(10,6))
#
plt.scatter(df['model_size_mb'],df['energy_kwh'],c=df['accuracy'],cmap='viri
dis')
# plt.colorbar()
# plt.xlabel('Model Size')
# plt.ylabel('Energy')
# plt.title('Energy vs Size')
# plt.show()

# ✓ DOBRY PRZYKŁAD: Czytelny, dobrze skomentowany kod
fig, ax = plt.subplots(figsize=(12, 6))

# Scatter plot z kolorem reprezentującym accuracy
scatter = ax.scatter(
    x=df['model_size_mb'],
    y=df['energy_kwh'],
    c=df['accuracy'],
    cmap='viridis',
    s=100,
    alpha=0.6,
    edgecolors='black',
    linewidth=0.5
)

# Dodanie colorbar z opisem
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Accuracy modelu', fontsize=12)

# Etykiety i tytuł
```

```
ax.set_xlabel('Rozmiar modelu (MB)', fontsize=12)
ax.set_ylabel('Zużycie energii (kWh)', fontsize=12)
ax.set_title('Zależność między rozmiarem modelu a zużyciem energii\n(kolor =
dokładność)',
             fontsize=14, fontweight='bold')

# Grid dla lepszej czytelności
ax.grid(True, alpha=0.3, linestyle='--')

plt.tight_layout()
# save_figure(fig, 'energy_vs_model_size.png')
plt.show()

print("\n✓ Wizualizacja pokazuje: większe modele zużywają więcej energii")
print("✓ Ale accuracy często rośnie z rozmiarem modelu")
print("✓ Trade-off: dokładność vs. zużycie energii!")
```

Tip

Zasady czytelnego kodu:

- Opisowe nazwy zmiennych (nie `x`, `y`, `df1`, tylko `energy_consumption`, `model_accuracy`)
- Komentarze wyjaśniające "dlaczego", nie "co"
- Jedna komórka = jedna logiczna operacja
- Wizualizacje z tytułami, etykietami osi i legendami

Przykład 3: Wersjonowanie i reprodukowalność

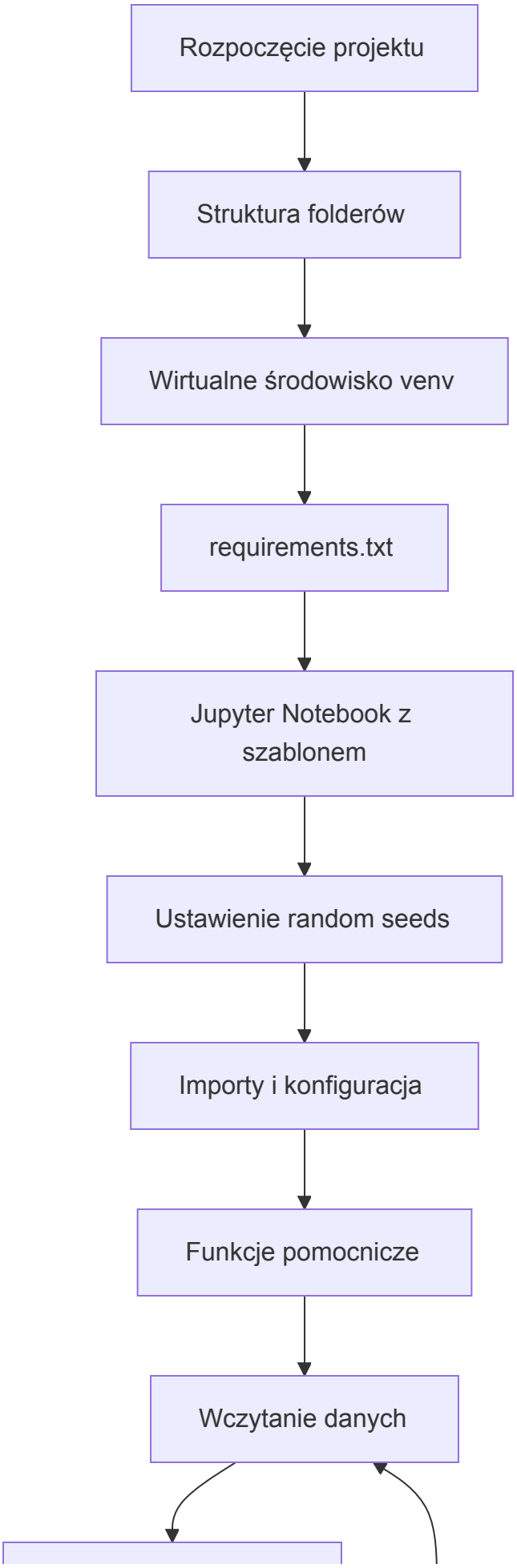
```
# === REPRODUKOWALNOŚĆ WYNIKÓW ===

# 1. ZAWSZE ustawiaj random seed dla reprodukowalności
import random

RANDOM_STATE = 42

# Numpy
np.random.seed(RANDOM_STATE)

# Python random
random.seed(RANDOM_STATE)

# Scikit-learn automatycznie używa parametru random_state
```

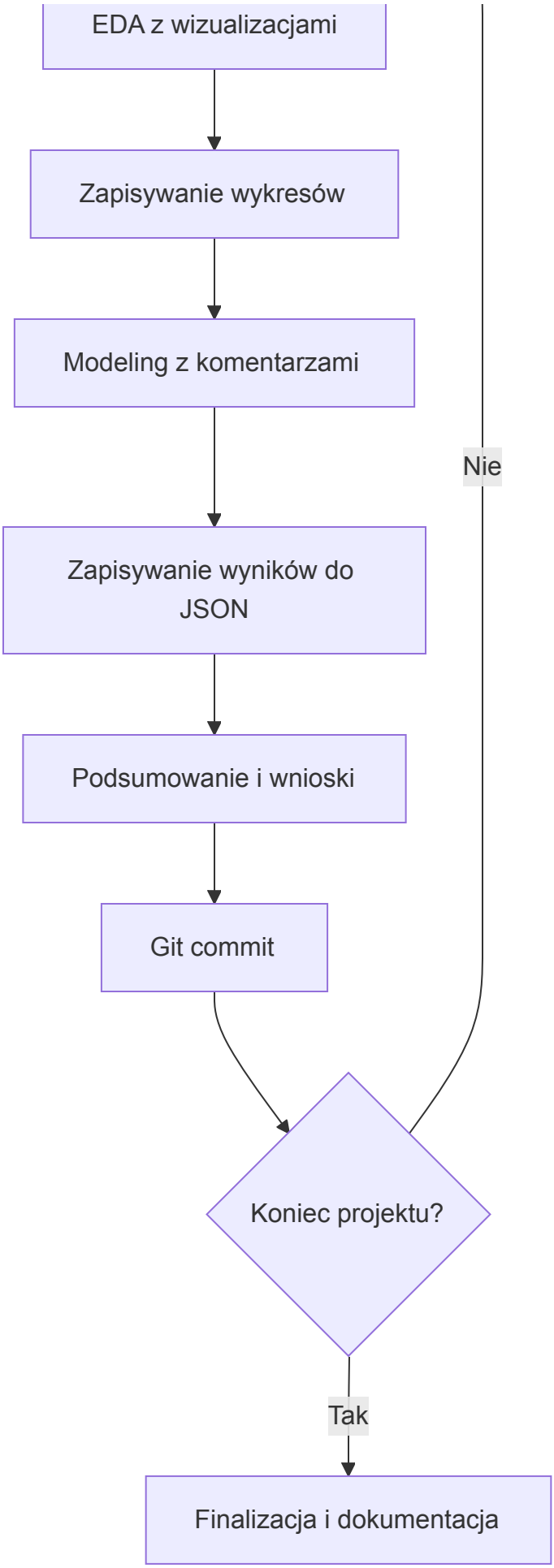
```python
# PyTorch
# torch.manual_seed(RANDOM_STATE)

# TensorFlow
# tf.random.set_seed(RANDOM_STATE)

print("✓ Random seeds ustawione - wyniki będą reprodukowalne!")

# 2. Zapisz informacje o środowisku
import platform

environment_info = {
    'python_version': sys.version,
    'platform': platform.platform(),
    'numpy_version': np.__version__,
    'pandas_version': pd.__version__,
    'sklearn_version': sklearn.__version__,
    'date_executed': datetime.now().strftime('%Y-%m-%d %H:%M:%S')
}

print("\n=== INFORMACJE O ŚRODOWISKU ===")
for key, value in environment_info.items():
    print(f"{key}: {value}")

# 3. Zapisz konfigurację eksperymentu
experiment_config = {
    'random_state': RANDOM_STATE,
    'test_size': TEST_SIZE,
    'model_type': 'LinearRegression',
    'features': ['model_size_mb', 'training_time_min', 'cpu_usage_percent'],
    'target': 'energy_kwh'
}

# Zapisz do pliku JSON
import json

config_path = 'experiment_config.json'
with open(config_path, 'w') as f:
    json.dump({**environment_info, **experiment_config}, f, indent=2)

print(f"\n✓ Konfiguracja eksperymentu zapisana: {config_path}")
```

✓ **Success**

Reprodukowalność to klucz w Data Science! Inni (i Ty za miesiąc) muszą być w stanie odtworzyć Twoje wyniki.

Schemat dobrych praktyk


Rozpoczęcie projektu
Struktura folderów
Wirtualne środowisko venv
requirements.txt
Jupyter Notebook z szablonem
Ustawienie random seeds
Importy i konfiguracja
Funkcje pomocnicze
Wczytanie danych


EDA z wizualizacjami
Zapisywanie wykresów
Modeling z komentarzami
Zapisywanie wyników do JSON
Podsumowanie i wnioski
Git commit
Koniec projektu?
Nie
Tak
Finalizacja i dokumentacja

🌱 Aspekt środowiskowy

W tej lekcji zwracamy szczególną uwagę na efektywne wykorzystanie zasobów obliczeniowych:

Optymalizacja zużycia energii w środowisku pracy

```python
# === GREEN IT W PRAKTYCE ===

import psutil
import time

class ResourceMonitor:
    """
    Klasa do monitorowania zużycia zasobów podczas pracy
    """
    def __init__(self):
        self.start_time = None
        self.start_cpu = None
        self.start_ram = None

    def start(self):
        """Rozpoczyna monitoring"""
        self.start_time = time.time()
        self.start_cpu = psutil.cpu_percent(interval=1)
        self.start_ram = psutil.virtual_memory().used / (1024**3)
        print("🌱 Monitoring zasobów rozpoczęty...")

    def stop(self):
        """Zatrzymuje monitoring i pokazuje statystyki"""
        if self.start_time is None:
            print("❌ Najpierw uruchom monitoring przez .start()")
            return

        end_time = time.time()
        end_cpu = psutil.cpu_percent(interval=1)
        end_ram = psutil.virtual_memory().used / (1024**3)

        duration = end_time - self.start_time
        avg_cpu = (self.start_cpu + end_cpu) / 2
        ram_used = end_ram - self.start_ram

        print("\n=== RAPORT ZUŻYCIA ZASOBÓW ===")
        print(f"⏱️  Czas wykonania: {duration:.2f} sekund")
        print(f"💻 Średnie użycie CPU: {avg_cpu:.1f}%")
        print(f"🧠 Zużycie RAM: {ram_used:.2f} GB")
```

```
        # Estymacja zużycia energii (uproszczona)
        # Zakładamy średnie zużycie procesora: 65W przy 100% CPU
        estimated_energy_wh = (avg_cpu / 100) * 65 * (duration / 3600)

        print(f"⚡ Szacowane zużycie energii: {estimated_energy_wh:.4f} Wh")

        if estimated_energy_wh < 1:
            print("✅ Efektywne zużycie energii!")
        else:
            print("⚠️  Wysokie zużycie - rozważ optymalizację")

# Przykład użycia
monitor = ResourceMonitor()

monitor.start()

# Symulacja obliczeń
data = np.random.rand(1000, 1000)
result = np.linalg.inv(data @ data.T)  # Obliczenia macierzowe

monitor.stop()
```

Praktyczne wskazówki Green IT

```
# === PORADY DLA EFEKTYWNEJ PRACY ===

green_it_tips = {
    '1. Próbkowanie danych':
        'Podczas testowania kodu używaj małych próbek danych (np. 1%
datasetu)',

    '2. Zamykanie sesji':
        'W Google Colab: Runtime → Disconnect. W Jupyter: File → Close and
Halt',

    '3. Wybór runtime':
        'Używaj GPU tylko gdy rzeczywiście potrzebne (Deep Learning). CPU
wystarczy do Pandas/NumPy',

    '4. Batch processing':
        'Przetwarzaj dane w częściach zamiast ładować cały dataset do RAM',

    '5. Czyszczenie pamięci':
        'Używaj del dla dużych obiektów, które nie są już potrzebne',
```

```
    '6. Efektywne biblioteki':
        'NumPy/Pandas są zoptymalizowane - używaj ich zamiast pętli Python',

    '7. Monitorowanie':
        'Regularnie sprawdzaj zużycie zasobów (htop, Task Manager, psutil)'
}

print("=== 🌱 GREEN IT W DATA SCIENCE ===\n")
for tip, description in green_it_tips.items():
    print(f"{tip}")
    print(f"   → {description}\n")

# Przykład: Efektywne czyszczenie pamięci
large_array = np.random.rand(10000, 10000)  # ~800 MB
print(f"Pamięć przed usunięciem: {psutil.virtual_memory().used /
(1024**3):.2f} GB")

del large_array  # Usuwamy dużą tablicę
import gc
gc.collect()  # Wymuszamy garbage collection

print(f"Pamięć po usunięciu: {psutil.virtual_memory().used / (1024**3):.2f}
GB")
print("✓ Pamięć zwolniona!")
```

> ⚠ **Warning**
>
> **Koszty środowiskowe Cloud Computing**:
>
> - Trenowanie GPT-3 wygenerowało ~500 ton CO_2 (ekwiwalent 5 samochodów przez rok)
> - Każda sesja Google Colab z GPU zużywa ~0.5-1 kWh energii
> - Darmowe zasoby = wspólne zasoby. Używaj odpowiedzialnie!

Wizualizacja zużycia energii

```
# === PORÓWNANIE ZUŻYCIA ENERGII: LOCAL vs CLOUD ===

import matplotlib.pyplot as plt

scenarios = ['Local CPU\n(laptop)', 'Local GPU\n(desktop)',
             'Colab CPU', 'Colab GPU', 'Colab TPU']
energy_wh = [30, 200, 50, 250, 400]  # Szacunkowe zużycie na godzinę
```

```
co2_g = [15, 100, 25, 125, 200]  # Emisja CO2 (gramy/godzinę)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Wykres 1: Zużycie energii
colors = ['green', 'orange', 'lightblue', 'orange', 'red']
bars1 = ax1.bar(scenarios, energy_wh, color=colors, edgecolor='black',
linewidth=1.5)
ax1.set_ylabel('Zużycie energii (Wh/godzina)', fontsize=12)
ax1.set_title('Zużycie energii różnych środowisk', fontsize=14,
fontweight='bold')
ax1.grid(axis='y', alpha=0.3)

# Dodanie wartości na słupkach
for bar in bars1:
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.0f} Wh',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

# Wykres 2: Emisja CO2
bars2 = ax2.bar(scenarios, co2_g, color=colors, edgecolor='black',
linewidth=1.5)
ax2.set_ylabel('Emisja CO₂ (g/godzina)', fontsize=12)
ax2.set_title('Emisja CO₂ różnych środowisk', fontsize=14,
fontweight='bold')
ax2.grid(axis='y', alpha=0.3)

# Dodanie wartości na słupkach
for bar in bars2:
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.0f} g',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.show()

print("\n🌱 WNIOSKI:")
print("✓ Local CPU (laptop) jest najbardziej energooszczędny")
print("✓ TPU zużywa najwięcej energii - używaj tylko gdy konieczne")
print("✓ Wybór odpowiedniego środowiska może zmniejszyć emisję CO₂ nawet o
90%!")
```

🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ Zadanie 1 – Pierwsze kroki w Google Colab

Utwórz nowy notatnik w Google Colab i napisz kod, który:

- Wyświetli informacje o systemie operacyjnym
- Sprawdzi dostępną RAM
- Sprawdzi, czy GPU jest dostępne

Wymagania:

- Użyj bibliotek: `platform`, `psutil`
- Wyświetl wyniki w czytelnym formacie

(proste)

2. ✏️ Zadanie 2 – Montowanie Google Drive

W Google Colab:

- Zamontuj Google Drive
- Utwórz folder `DS_Course/Lesson02`
- Zapisz do tego folderu plik tekstowy z Twoim imieniem i datą

Wymagania:

- Sprawdź, czy folder został utworzony
- Wyświetl zawartość folderu

(proste)

3. ✏️ Zadanie 3 – Instalacja bibliotek

W Google Colab zainstaluj biblioteki:

- `seaborn` (wizualizacja)
- `plotly` (interaktywne wykresy)
- `pandas-profiling` (automatyczne raporty)

Wymagania:

- Po instalacji sprawdź wersje każdej biblioteki

- Zaimportuj je i wyświetl wersję

(proste)

4. ✏️ Zadanie 4 – Utworzenie venv lokalnie

Na swoim komputerze (nie Colab):

- Utwórz folder `datascience_project`
- Stwórz wirtualne środowisko `ds_env`
- Aktywuj je i zainstaluj `jupyter`, `pandas`, `numpy`
- Wygeneruj `requirements.txt`

Wymagania:

- Zrób screenshot terminala pokazujący aktywne venv
- Pokaż zawartość `requirements.txt`

(proste)

5. ✏️ Zadanie 5 – Struktura projektu

Utwórz strukturę folderów dla projektu analizy sprzedaży:

- `sales_analysis/`
- Podfoldery: `data/raw`, `data/processed`, `notebooks`, `src`, `models`, `reports`

Wymagania:

- Użyj funkcji z lekcji lub stwórz własną
- Dodaj `README.md` z opisem projektu
- Dodaj `.gitignore` ignorujący `data/raw/`

(proste)

6. ✏️ Zadanie 6 – Pierwszy notatnik z szablonem

Utwórz Jupyter Notebook o nazwie `lesson02_practice.ipynb`:

- Sekcja 1: Importy (pandas, numpy, matplotlib)
- Sekcja 2: Konfiguracja (ustawienie random seed na 42)
- Sekcja 3: Utworzenie przykładowego datasetu (100 wierszy, 3 kolumny)
- Sekcja 4: Podstawowa eksploracja (info, describe, head)

Wymagania:

- Użyj struktury z przykładów
- Każda sekcja w osobnej komórce z komentarzem nagłówkowym

(proste)

7. ✏️ Zadanie 7 – Monitoring zasobów

Napisz kod, który:

- Monitoruje zużycie CPU i RAM przed i po wykonaniu obliczeń
- Wykonaj obliczenie: stwórz macierz 1000x1000 i policz jej odwrotność
- Wyświetl raport zużycia zasobów

Wymagania:

- Użyj biblioteki `psutil`
- Wyświetl różnicę w zużyciu RAM

(proste)

8. ✏️ Zadanie 8 – Zapisywanie wykresów

Utwórz prosty wykres słupkowy pokazujący zużycie energii 3 urządzeń:

- Laptop: 30 Wh
- Desktop: 150 Wh
- Server: 400 Wh

Wymagania:

- Użyj `matplotlib`
- Dodaj tytuł, etykiety osi, kolory
- Zapisz wykres jako `energy_comparison.png`

(proste)

✏️ Zadania średnie (9-12)

9. ✏️ Zadanie 9 – Porównanie Colab vs Local

Stwórz tabelę porównawczą (DataFrame) Google Colab vs Jupyter Notebook lokalnie:

Kolumny: Feature, Google Colab, Jupyter Local

Wiersze:

- Koszt
- Dostęp do GPU
- Praca offline
- Limity czasowe
- Kontrola środowiska

Wymagania:

- Użyj Pandas DataFrame
- Wyeksportuj do CSV
- Bonus: Stwórz wizualizację pokazującą różnice

(średnie)

10. ✏️ Zadanie 10 – Analiza zużycia zasobów

Utwórz dataset symulujący zużycie zasobów przez 5 różnych modeli ML:

- Kolumny: model_name, training_time_min, cpu_percent, ram_gb, energy_kwh
- 50 wierszy (10 pomiarów na model)

Wymagania:

- Oblicz średnie zużycie dla każdego modelu
- Stwórz wykres pokazujący, który model jest najbardziej energooszczędny
- Napisz wnioski w markdown cell

(średnie)

11. ✏️ Zadanie 11 – Funkcja do automatycznego raportu środowiska

Napisz funkcję `generate_environment_report()`, która:

- Zbiera informacje o środowisku (Python version, OS, zainstalowane biblioteki)
- Zbiera informacje o zasobach (CPU, RAM, GPU jeśli dostępne)
- Zapisuje raport do pliku JSON z timestampem

Wymagania:

- Funkcja przyjmuje parametr `output_dir`
- JSON zawiera przynajmniej 8 informacji
- Funkcja zwraca ścieżkę do zapisanego pliku

(średnie)

12. ✏️ Zadanie 12 – Green IT Calculator

Stwórz kalkulator śladu węglowego dla sesji Data Science:

Input:

- Czas pracy (godziny)
- Typ urządzenia (laptop, desktop, cloud GPU)
- Region (różna emisja CO_2 zależnie od źródła energii)

Output:

- Zużycie energii w kWh
- Emisja CO_2 w gramach
- Porównanie do codziennych aktywności (np. "równoważne 5 km jazdy autem")

Wymagania:

- Funkcja z parametrami
- Wizualizacja wyników

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 Zadanie 13 – Automatyczny setup projektu

Stwórz skrypt Python, który automatycznie przygotowuje nowy projekt DS:

- Tworzy strukturę folderów
- Inicjalizuje git repo (`git init`)
- Tworzy venv i instaluje podstawowe pakiety
- Generuje template Jupyter Notebook
- Tworzy `README.md` i `.gitignore`

Wymagania:

- Skrypt przyjmuje nazwę projektu jako argument
- Wszystko w jednej funkcji `setup_project(project_name)`
- Raport z wykonanych kroków

(challenge)

14. 🧠 Zadanie 14 – Resource-aware notebook executor

Stwórz narzędzie, które:

- Uruchamia Jupyter Notebook
- Monitoruje zużycie zasobów w czasie rzeczywistym
- Jeśli zużycie przekroczy próg (np. 80% RAM), wysyła ostrzeżenie
- Generuje raport z wykresami zużycia zasobów po zakończeniu

Wymagania:

- Monitoring w osobnym wątku (threading)
- Wizualizacja zużycia w czasie (line plot)
- Eksport raportu do HTML

(challenge)

15. 🧠 Zadanie 15 – Colab-to-Local converter

Napisz narzędzie konwertujące notebook Google Colab na format lokalny:

- Usuwa Colab-specific code (montowanie Drive, instalacje)
- Aktualizuje ścieżki do plików (z `/content/drive/` na lokalne)
- Dodaje komórki z instalacją venv
- Generuje `requirements.txt` z pakietów używanych w notebooku

Wymagania:

- Parsowanie .ipynb (JSON)
- Detekcja Colab-specific cells
- Generowanie nowego .ipynb

(challenge)

16. 🧠 Zadanie 16 – Energy Benchmark Suite

Stwórz benchmark porównujący zużycie energii różnych operacji:

- Operacje NumPy vs Python loops
- Pandas operations vs iterrows
- Matplotlib vs Plotly (rendering time)
- Sklearn vs XGBoost (training)

Wymagania:

- Minimum 10 różnych operacji
- Pomiar czasu i zużycia zasobów dla każdej
- Wykres porównawczy
- Raport z rekomendacjami "co jest bardziej green"

(challenge)

17. 🧠 Zadanie 17 – Smart Dependency Manager

Stwórz narzędzie, które analizuje Jupyter Notebook i:

- Wykrywa wszystkie używane importy
- Sprawdza, które są zainstalowane, a które nie
- Generuje minimalny `requirements.txt` (tylko używane pakiety)
- Identyfikuje konfliktujące wersje
- Sugeruje alternatywy (np. lżejsze biblioteki)

Wymagania:

- Parsowanie kodu w cellach
- Sprawdzanie zainstalowanych pakietów (pkg_resources)
- Generowanie raportu w markdown

(challenge)

18. 🧠 Zadanie 18 – Multi-environment tester

Stwórz system testujący ten sam kod w różnych środowiskach:

- Local Python 3.9
- Local Python 3.11
- Google Colab
- Docker container

Wymagania:

- Automatyczne uruchamianie tego samego notatnika w każdym środowisku
- Porównanie wyników (czy są identyczne?)
- Porównanie czasu wykonania
- Raport z różnicami

(challenge)

19. 🧠 Zadanie 19 – Notebook Quality Checker

Stwórz narzędzie do audytu jakości Jupyter Notebooks:

Sprawdza:

- Czy są komentarze i markdown cells (min 20%)
- Czy używane są magic commands (%time, %matplotlib inline)
- Czy są ustawione random seeds
- Czy są zapisane wykresy
- Czy struktura odpowiada best practices
- Complexity score (długość komórek, zagnieżdżenie)

Output:

- Quality score (0-100)
- Lista rekomendacji do poprawy

(challenge)

20. 🧠 Zadanie 20 – Green Data Science Dashboard

Stwórz interaktywny dashboard (Plotly Dash lub Streamlit) do monitorowania ekologicznych aspektów pracy DS:

Features:

- Upload Jupyter Notebook
- Analiza zużycia zasobów (symulacja na bazie kodu)
- Obliczenie carbon footprint
- Porównanie z alternatywnymi podejściami
- Sugestie optymalizacji
- Wizualizacje (gauges, charts)
- Export raportu PDF

Wymagania:

- Interaktywny UI
- Obliczenia w czasie rzeczywistym
- Atrakcyjne wizualizacje
- Funkcjonalność export

(challenge)

📚 Podsumowanie

W tej lekcji poznaliśmy fundamenty środowiska pracy Data Scientist:

Czego się nauczyliśmy:

1. **Google Colab**:
   - Praca w chmurze bez instalacji
   - Dostęp do darmowych GPU/TPU
   - Montowanie Google Drive
   - Zarządzanie sesjami
2. **Jupyter Notebook lokalnie**:
   - Instalacja Pythona i Jupyter
   - Tworzenie wirtualnych środowisk (venv)
   - Zarządzanie pakietami (pip, requirements.txt)
3. **Organizacja projektów**:
   - Standardowa struktura folderów Data Science
   - Best practices w organizacji kodu
   - Reprodukowalność wyników (random seeds)
   - Wersjonowanie konfiguracji
4. **Dobre praktyki**:
   - Czytelna struktura notebooka
   - Komentarze i dokumentacja
   - Zapisywanie wykresów i wyników
   - Funkcje pomocnicze
5. **Green IT**:
   - Monitoring zużycia zasobów
   - Optymalizacja energii w ML
   - Świadome wybory (CPU vs GPU)
   - Zamykanie niepotrzebnych sesji

Powiązania z poprzednimi lekcjami:

- W **Lekcji 01** poznaliśmy różne typy zadań ML - teraz wiemy, **gdzie i jak** je rozwiązywać
- Zrozumieliśmy trade-off między dokładnością a zużyciem energii

Przygotowanie do następnych lekcji:

- W kolejnych lekcjach będziemy pracować z **prawdziwymi danymi** - teraz mamy gotowe środowisko
- Poznamy **NumPy i Pandas** - już wiemy, jak je zainstalować i organizować projekty
- Nauczymy się **Machine Learning** - środowisko jest skonfigurowane i gotowe

Kluczowe zasady na przyszłość:

✓ **Success**

Zawsze pamiętaj:

- Wirtualne środowiska dla każdego projektu
- Struktura folderów zgodna ze standardem
- Random seeds dla reprodukowalności
- Komentarze i dokumentacja
- Monitoring zasobów i świadome używanie energii
- Regularne zapisywanie pracy (Colab ma limity czasowe!)

📖 Dodatkowe Zasoby

Dokumentacja:

- Google Colab - Getting Started
- Jupyter Notebook Documentation
- Python Virtual Environments Guide
- pip Documentation

Tutorials:

- Real Python - Jupyter Notebook Tutorial
- Google Colab Tips and Tricks
- Cookiecutter Data Science - Project Structure

Narzędzia:

- nbconvert - Convert notebooks to other formats
- nbdime - Diff and merge for notebooks
- Jupyter Lab - Next generation Jupyter

Green IT Resources:

- Green Algorithms Calculator
- ML CO2 Impact
- Energy and Policy Considerations for Deep Learning

Datasets do praktyki:

- Google Colab sample datasets: `/sample_data/`
- Kaggle Datasets: https://www.kaggle.com/datasets
- UCI Machine Learning Repository: https://archive.ics.uci.edu/ml/

✅ **Gratulacje! Ukończyłeś Lekcję 02!**

Teraz masz profesjonalne środowisko pracy i znasz wszystkie narzędzia potrzebne Data Scientist. W następnej lekcji zapoznamy się z systemem kontroli wersji Git i GitHub, aby bezpiecznie zarządzać kodem i współpracować z innymi!

🌱 **Pamiętaj**: Każda zamknięta sesja Colab, każde wyłączone GPU, każda zoptymalizowana operacja to wkład w lepszą przyszłość naszej planety. Data Science może być green - zależy to od nas!